

作業用ファイル一覧と `chordmap.json` / `sections.json` / `lyric_anchors.json` への関連性

以下の表では、`composer4` レポジトリのうち、大幅な変更が加わる前の `tools`、`scripts`、`generator`、`utilities` ディレクトリにあるスクリプト群から、コードマップ (`chordmap.json`)・セクション定義 (`sections.json`)・リリックアンカー (`lyric_anchors.json`) の作成に役立つものを抜き出して解説します。

カテゴリ	ファイル名	概要と *.json 作成への応用
コードマップ (<code>chordmap.json</code>) 関連	<code>utilities/infer_chords_from_midi.py</code>	音源MIDIから基本コードを推定し、 <code>base_chords</code> を含むYAMLを生成する。 <code>music21</code> を使ってMIDIファイルを解析し、各小節の代表和音（最長持続の和音）を選び、キーの推定やローマ数字表記も記録する ¹ 。この出力は物語のベースとなるコード進行を <code>chordmap.json</code> に落とし込む際の土台になる。
	<code>utilities/chordmap_merge.py</code>	既存曲の ベースコード と物語や感情を表す ナラティブコード をマージして最終的なコードマップを生成するスクリプト。ルート変更の制限、テンション付加、ベース音の保護など様々なポリシーを適用しながら、ベースの進行に彩りを加える ² 。この結果を <code>chordmap.json</code> の <code>final_chords</code> として採用する。
	<code>scripts/emotion_to_chords.py</code>	物語の感情値（ヴァランス・アラウザル）とセクション属性から適切なコード進行を生成する。 <code>progression_templates.yaml</code> のテンプレートやLAMDaデータベースを参照し、候補進行を抽出し品質スコアでソートする ³ 。多様性フィルタリングにも対応し、同質化を抑えた候補を返す ³ ⁴ 。物語の情緒に合った進行を <code>chordmap.json</code> に反映する際に有用。
	<code>scripts/diversity_analyzer.py</code>	コード進行の多様性や類似度を計算・比較するツール。N-gram多様性スコアや進行同士の類似度を計算し、Top-K候補の多様性フィルタリングを行う ⁵ ⁶ 。 <code>emotion_to_chords.py</code> から呼び出され、同質化の抑制に役立つため、 <code>chordmap.json</code> に採用するコード進行選定の品質向上に利用できる。

カテゴリ	ファイル名	概要と *.json 作成への応用
	utilities/ ujam_from_chordmap.py	完成した <code>chordmap.final.yaml</code> とテンポマップから UJAM Sparkle 用のMIDIを生成するユーティリティ。コード進行をコードトラックとパルストラックに分けてMIDI出力する ⁷ 。直接 <code>chordmap.json</code> 生成には用いないが、生成したマップを音として確認する際に役立つ。
	generator/chord_voicer.py	<code>music21</code> を用いてコードシンボルをボイスシングし、実際のピッチ列に変換するクラス。コードラベルの正規化や三和音・テンションの解析などを含む ⁸ ⁹ 。 <code>chordmap.json</code> のコードを具体的な和音として鳴らす際に参考になるが、マップ生成そのものには直接使用しない。
セクション定義 (sections.json) 関連	tools/compare_sections.py	コードマップ (YAML) と構成設定ファイル (<code>config/main_cfg.yaml</code> など) の各セクション名を比較し、どちらか片方にしか存在しないセクションを一覧表示する ¹⁰ 。セクション名の統一漏れを検出するのに便利で、 <code>sections.json</code> の整合性チェックに活用できる。
	utilities/ section_validator.py	セクション定義YAMLに記載された各セクションのバー範囲が、対応するMIDIファイルの実際のバー範囲と一致しているかを検証するユーティリティ。存在しないラベルやバー範囲の不一致を検出し、エラーを投げる ¹¹ ¹² 。 <code>sections.json</code> で指定したセクションの長さがMIDIの長さ合っているかを確認する際に役立つ。
	utilities/ core_music_utils.py (補助)	<code>ChordVoicer</code> などから呼び出され、拍子記号のオブジェクト生成などの音楽的ユーティリティを提供している。セクション解析やコード配置で拍子情報が必要な場合に利用できるが、直接 <code>sections.json</code> を生成する機能は持たない。
リリックアンカー (lyric_anchors.json) 関連	scripts/align_lyrics.py	音声ファイルとMIDIファイルを用いて歌詞の音素ごとのタイミングを求めるスクリプト。PyTorch による事前学習済みモデルを読み込み、音声から音素を推定し、MIDIのオンセットをフレームにマッピングすることで、各音素の出現時刻をミリ秒単位で出力する ¹³ 。この出力を整形して <code>lyric_anchors.json</code> に記録することで、歌詞の音符との同期情報を構築できる。

カテゴリ	ファイル名	概要と *.json 作成への応用
	scripts/ train_lyrics_align.py	歌詞アラインメントモデルの学習用スクリプト。音声・MIDI・音素ラベルを含むデータセットを読み込み、PyTorch Lightning を使って CTC ベースのモデルを訓練する ¹⁴ 。新しい言語や歌手向けにアラインメント性能を向上させたい場合に役立つが、既存モデルを利用するだけなら必須ではない。

まとめ

上記のスクリプトは、物語の感情やMIDIデータからコード進行を生成・マージしたり、進行の多様性を評価して `chordmap.json` に組み込むのに役立ちます。またセクション情報の整合性チェックや歌詞と音符の同期推定を行うツールも含まれており、`sections.json` や `lyric_anchors.json` を作成する際の補助になるでしょう。これらのスクリプトを活用することで、音楽作品制作の自動化を効率的に進められるはずです。

- 1 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/utilities/infer_chords_from_midi.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/utilities/infer_chords_from_midi.py
- 2 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/utilities/chordmap_merge.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/utilities/chordmap_merge.py
- 3 4 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/scripts/emotion_to_chords.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/scripts/emotion_to_chords.py
- 5 6 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/scripts/diversity_analyzer.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/scripts/diversity_analyzer.py
- 7 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/utilities/ujam_from_chordmap.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/utilities/ujam_from_chordmap.py
- 8 9 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/generator/chord_voicer.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/generator/chord_voicer.py
- 10 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/tools/compare_sections.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/tools/compare_sections.py
- 11 12 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/utilities/section_validator.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/utilities/section_validator.py
- 13 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/scripts/align_lyrics.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/scripts/align_lyrics.py
- 14 [raw.githubusercontent.com](https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/scripts/train_lyrics_align.py)
https://raw.githubusercontent.com/kinoshitayoshihiro/composer4/main/scripts/train_lyrics_align.py